

SQL

CREATE TABLE

```
CREATE TABLE table_name (  
    column1 datatype constraint,  
    column2 datatype constraint,  
    column3 datatype constraint,  
    CONSTRAINT name_constr PRIMARY KEY(column...),  
    CONSTRAINT name_constr FOREIGN KEY(column...)  
        REFERENCES table_name (column)  
);
```

CREATE TABLE desde una existente

```
CREATE TABLE new_table AS  
SELECT column1, column2,...  
FROM existing_table  
WHERE condition;
```

INSERT INTO

- **INSERT INTO** table_name (column1, column2, column3, ...)
- **VALUES** (value1, value2, value3, ...);

SELECT

SELECT column1, column2, ...

FROM table_name

WHERE condition

ORDER BY column;

UPDATE

UPDATE table_name

SET column1 = value1, column2 = value2, ...

WHERE condition;

DELETE

- **DELETE FROM** table_name **WHERE** condition;

DELETE TABLE

- **DROP TABLE** table_name;

Tipos de datos

Los tipos de datos disponibles se deben consultar en el manual del SGBD relacional que se esté utilizando.

Entero

SQL	SQLite
INT INTEGER TINYINT SMALLINT MEDIUMINT BIGINT UNSIGNED BIG INT INT2 INT8	INTEGER

Texto y Blob

SQL	SQLite
CHARACTER(20) VARCHAR(255) VARYING CHARACTER(255) NCHAR(55) NATIVE CHARACTER(70) NVARCHAR(100) TEXT CLOB	TEXT
BLOB	BLOB

Real y Numérico

SQL	SQLite
REAL DOUBLE DOUBLE PRECISION FLOAT	REAL
NUMERIC DECIMAL(10,5) BOOLEAN DATE DATETIME	NUMERIC

Operadores Lógicos

a	b	a AND b	A OR b	NOT b
True	True	True	True	False
True	False	False	True	True
True	Null	Null	True	Null
False	False	False	False	
False	Null	False	Null	
Null	Null	Null	Null	

Operadores de comparación

Operador	Equivalencia
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
=	Igual que
<> !=	Distinto de
'a BETWEEN x AND y	'a >= x AND a <= y
'a NOT BETWEEN x AND y	'a < x OR a > y
'a IS NULL	True si a es nulo
'a IS NOT NULL	True si a es no nulo
'a IN (v1, v2, ...)	'a = v1 OR a = v2 OR ...

Operadores matemáticos

Operador	Significado (ejemplo)
+	Suma
-	Resta
*	Multiplicación
/	División
%	Resto
^	Potencia ($3^2 = 9$)

Funciones matemáticas

Funciones	Significado
ABS(x)	Valor absoluto de x
SIGN(x)	Devuelve el signo de x (-1 o 1)
MOD(x,y)	Resto de la division entera
SQRT(x)	Raíz cuadrada de x
CBRT(x)	Raíz cubica de x
CEIL(x)	Entero más cercano por debajo de x
FLOOR(x)	Entero más cercano por encima de x
ROUND(x)	Redondea al entero más cercano
ROUND(x,n)	Redondeo x a n dígitos decimales
TRUNC(x)	Trunca x

Operadores y funciones de texto

Operador	Significado
'texto texto	Concatena 2 textos
'texto LIKE expr	Devuelve TRUE sigue el patrón de expr
LENGTH(texto)	Número de caracteres del texto
GLOB(texto) (* y ?)	Es la función del estándar equivalente a LIKE

Operadores y funciones de texto

Operador	Significado
SUBSTR(texto, n, long)	Devuelve un subtexto que empieza en la posición n de largo long
LOWER(texto)	Devuelve el texto en minúsculas
UPPER(texto)	Devuelve el texto en mayúsculas
TRIM(texto)	Elimina espacios delante y detrás del texto
REPLACE(texto, buscar, nuevo)	Reemplaza todas las ocurrencias de un subtexto por otro
INSTR(texto, subtexto)	Devuelve la posición de la primera aparición del subtexto. Si no se encuentra, devuelve 0

Operaciones con fechas

SQLite no tiene un tipo de dato dedicado para fechas; estas se almacenan como cadenas en formato ISO-8601 (**AAAA-MM-DD HH:MM:SS**)

Expresión CASE

La expresión CASE es una estructura condicional (tipo if-else o switch), evalúa condiciones y devuelve un valor específico cuando una de ellas se cumple. Se puede utilizar en cláusulas como SELECT, WHERE, ORDER BY y UPDATE

```
SELECT ProductName,  
       UnitPrice,  
       CASE  
         WHEN UnitPrice > 100 THEN 'Premium'  
         WHEN UnitPrice BETWEEN 50 AND 100 THEN 'Estándar'  
         ELSE 'Económico'  
       END AS categoria  
FROM Products;
```

Funciones de columna

Función	Significado
COUNT(*)	Cuenta filas
COUNT(columna)	Cuenta valores no nulos
SUM(columna)	Suma los valores de la columna
MAX(columna)	Obtiene el valor máximo de la columna
MIN(columna)	Obtiene el valor mínimo de la columna
AVG(columna)	Obtiene el promedio de la columna

Cláusula GROUP BY

- Forma **grupos** con las filas que tienen en común los valores de una o varias columnas. Sobre cada grupo se pueden aplicar las funciones de columna (SUM, MAX, MIN, AVG, COUNT)

SELECT columna1, **COUNT**(*)

FROM Tabla

WHERE Condición

GROUP BY columna1;

Cláusula HAVING

La cláusula **HAVING**, que puede aparecer tras **GROUP BY**, se utilizan las funciones de grupo para hacer restricciones sobre los grupos que se han formado.

SELECT columna , columna

FROM Tabla

WHERE condición

GROUP BY columna

HAVING condición_para_el_grupo

ORDER BY columna